

Architecture & security review package

Diagrams and inventories for security/ATO review of the Claude apps gateway deployment (AWS GovCloud `us-gov-west-1`). Everything here is generated from the templates in `cloudformation/` at the commit this file ships with — when the code and this document disagree, the code wins and this file needs a PR.

The diagrams are hand-laid-out SVGs under `docs/diagrams/`, generated by `docs/diagrams/generate.py` (edit the script, re-run it, and commit both). SVGs print cleanly and embed directly in submission documents; for PNG exports: `python3 -c "import cairosvg; cairosvg.svg2png(url='docs/diagrams/<name>.svg', write_to='<name>.png', scale=2)".`

Contents:

1. [System architecture & trust boundaries](#)
2. [Network flows, ports & TLS state](#)
3. [Developer authentication \(Okta OIDC\)](#)
4. [Database credential lifecycle](#)
5. [Telemetry & audit data flows](#)
6. [Secrets & key inventory](#)
7. [Security-group matrix](#)
8. [Stack dependencies & deploy order](#)
9. [Data-at-rest encryption map](#)
10. [Known accepted risks / review notes](#)
11. [Appendix — combined system view](#)

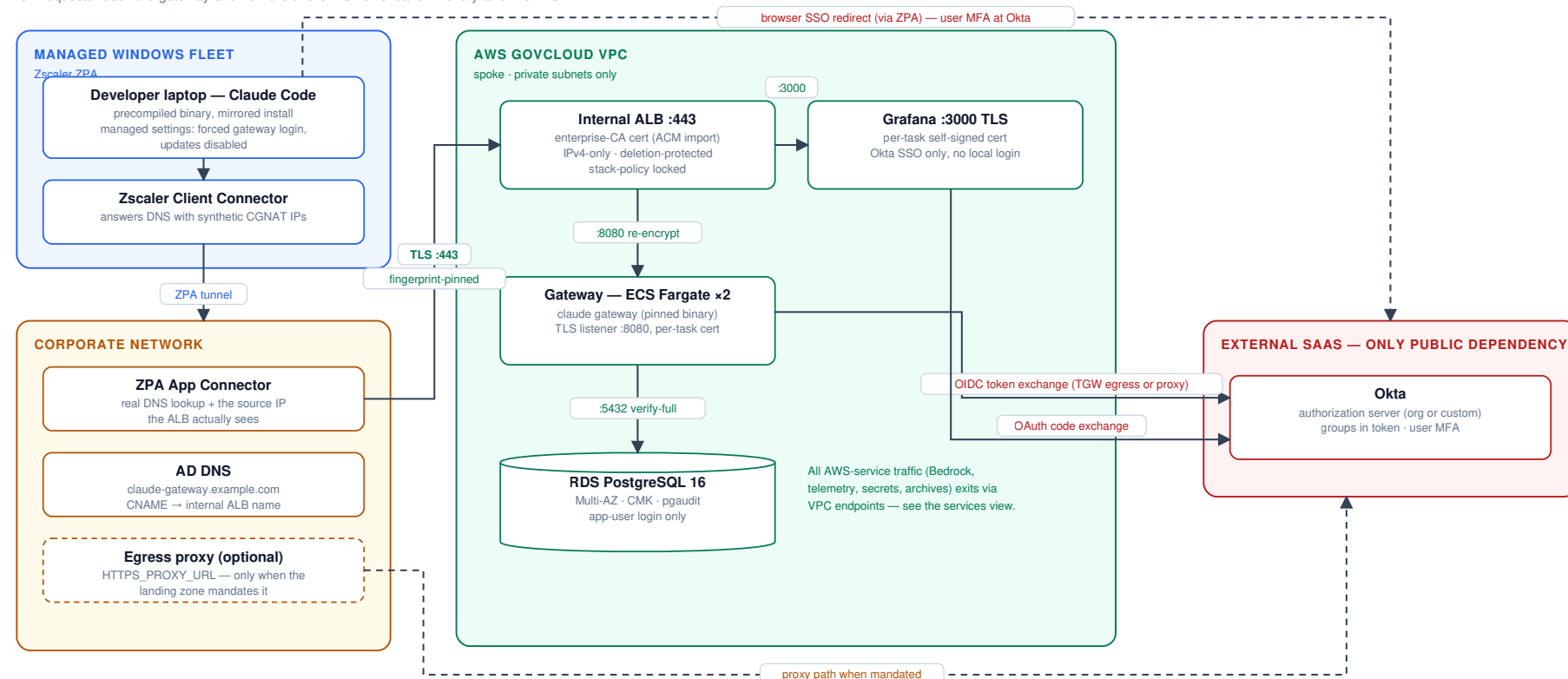
1. System architecture & trust boundaries

Four trust zones: the managed Windows fleet (Zscaler ZPA), the corporate network (App Connectors + AD DNS + egress proxy), the AWS GovCloud VPC (all workload components), and external SaaS (Okta — the single public dependency; Bedrock and all other AWS services are reached over private endpoints or the AWS backbone). Two complementary views; the combined single-page version is in the [appendix](#).

Access & network path — how requests reach the gateway and how the OIDC flows leave:

Access & network path

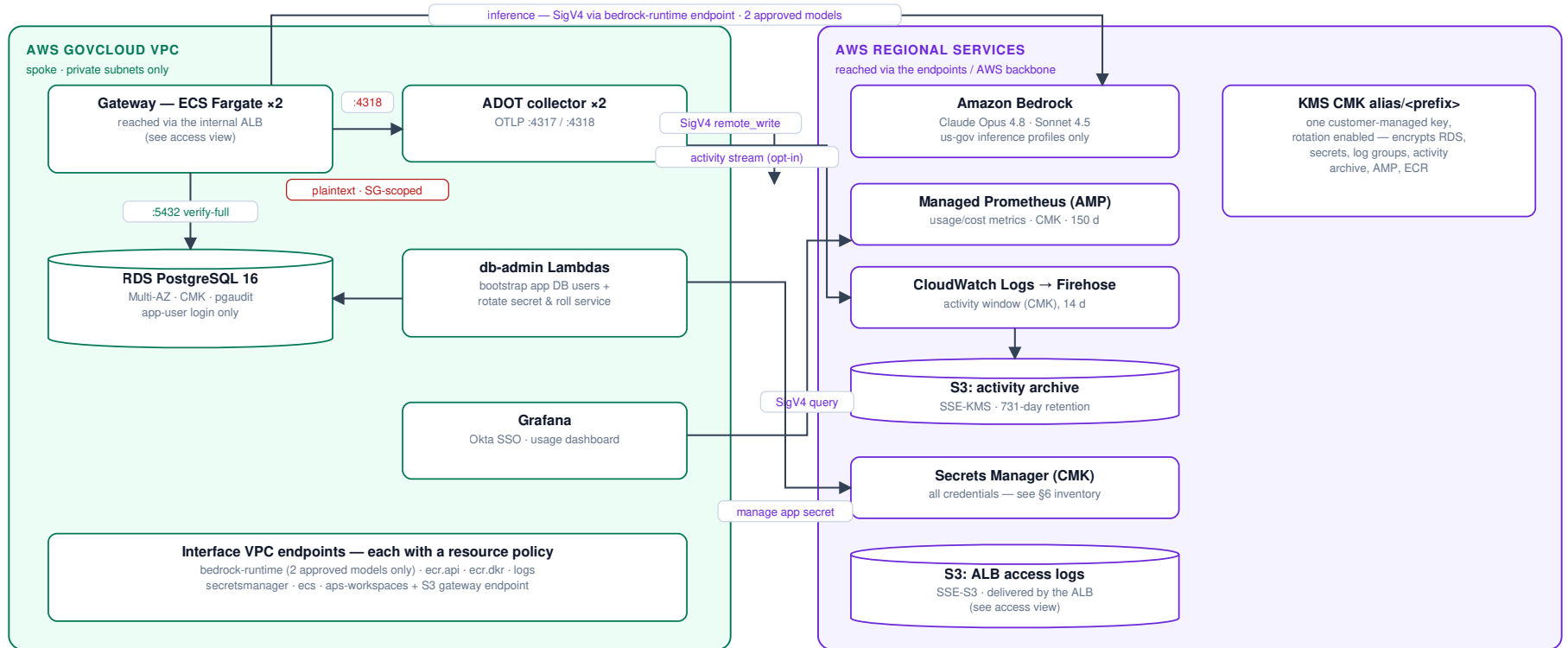
how requests reach the gateway and how the two OIDC flows leave — every arrow is TLS



Workloads & AWS data services — inference, telemetry, secrets and encryption inside AWS:

Workloads & AWS data services

inference, telemetry, secrets and encryption — developer access path is in the access view



The gateway→collector OTLP hop is the only unencrypted flow (SG-to-SG scoped — accepted risk §10). Okta egress and the developer path are on the access view.

Secrets are injected into tasks at launch by ECS (execution roles hold GetSecretValue + kms:Decrypt on exactly their own secrets — see §6).

Key boundary statements for the reviewer:

- **No public ingress.** The ALB is internal; reaching it requires the ZPA app segment (or VPN) and an allow-listed connector source CIDR.
- **No public egress from the inference path.** Bedrock is reached via an interface endpoint whose policy allows exactly the two approved models; WebSearch is disabled on gateway sessions.
- **One public dependency:** the Okta issuer (OIDC has no VPC endpoint). Egress rides TGW central NAT or the mandated corporate proxy, port-scoped by security groups.

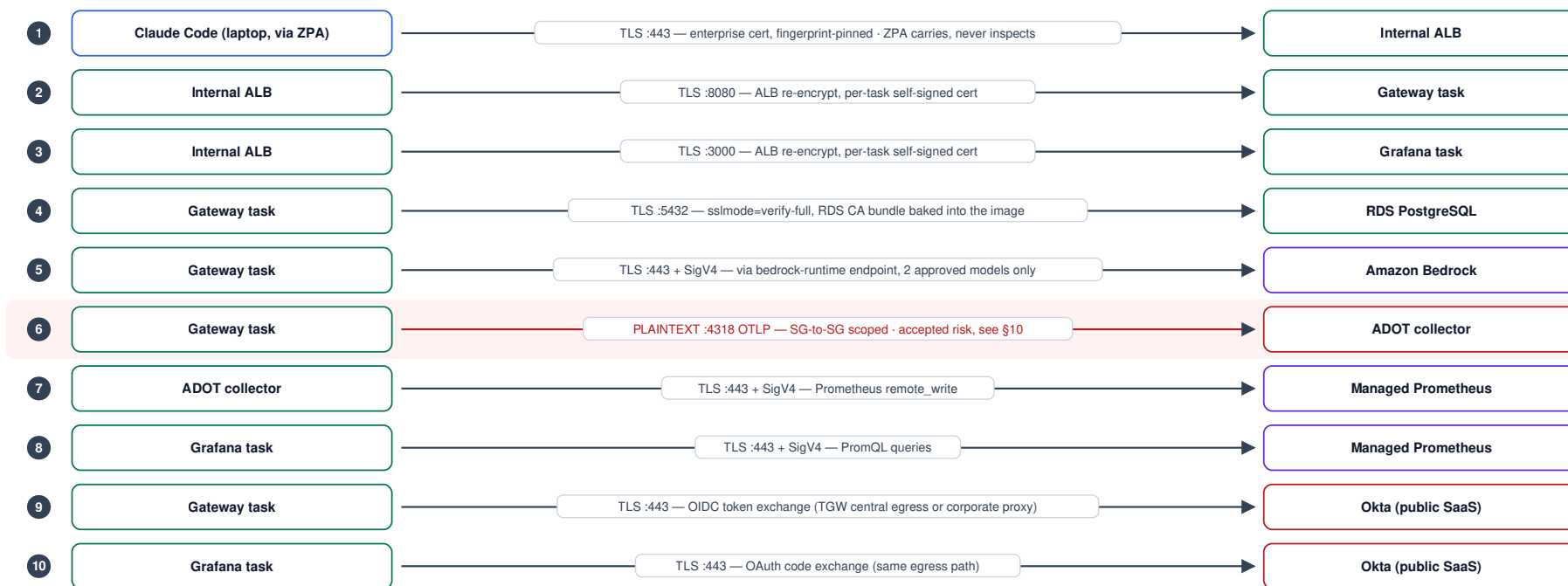
- **Client fleet never contacts Anthropic:** binaries are mirrored, checksum- and GPG-verified, and distributed from an internal share; all update paths are disabled by managed settings.

2. Network flows, ports & TLS state

Every hop, its port, protocol, and where the TLS session terminates.

Network flows, ports & TLS state — every hop in the system

one row per flow; row 6 is the only unencrypted hop



TLS termination points: developer→ALB terminates on the enterprise cert (developers pin its fingerprint); ALB→task hops terminate on per-task ephemeral certs (ALBs do not validate target certs; keys never leave the task).

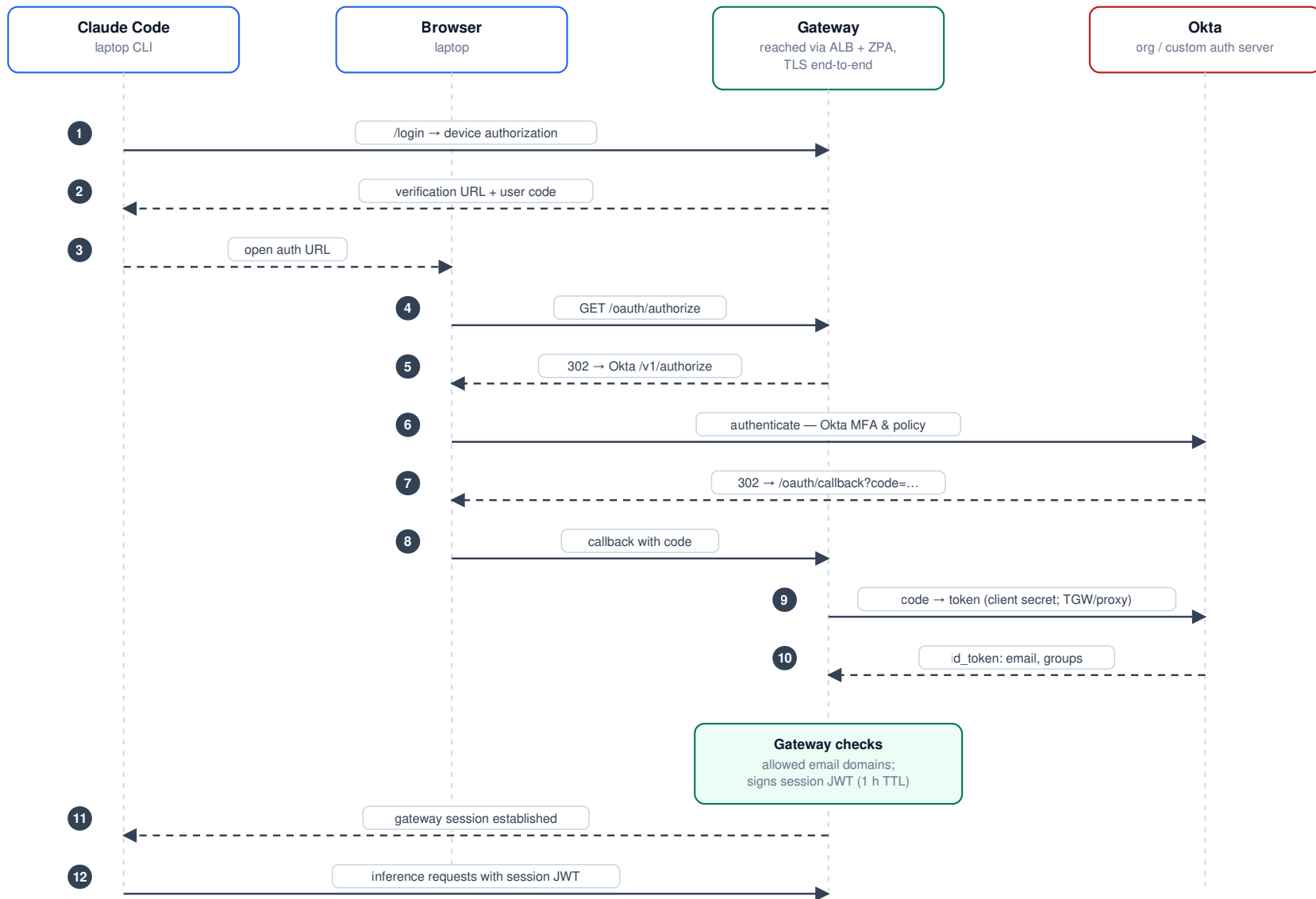
All AWS-service hops (5, 7, 8) also carry SigV4 request signing on top of TLS. DNS to the VPC resolver is exempt from security-group evaluation (AWS platform behavior).

#	Hop	Port	Encryption	Server identity proven by
1	Laptop → ALB	443	TLS (FIPS policy)	Enterprise-CA cert; client pins SHA-256 fingerprint on first connect
2	ALB → gateway	8080	TLS	Per-task ephemeral cert (ALB does not validate targets; key never leaves task)
3	ALB → Grafana	3000	TLS	Per-task ephemeral cert (same model)
4	Gateway/Lambda → RDS	5432	TLS <code>verify-full</code>	RDS regional CA bundle baked into images; <code>rds.force_ssl=1</code> server-side
5	Gateway → Bedrock	443	TLS + SigV4	AWS cert; endpoint policy = 2 approved models only
6	Gateway → collector	4317–4318	Plaintext	n/a — SG-to-SG scoped; see §10
7–8	Collector/Grafana → AMP	443	TLS + SigV4	AWS cert; endpoint policy = this workspace only
9–10	Gateway/Grafana → Okta	443	TLS	Public CA; only internet-bound flow

3. Developer authentication (Okta OIDC)

Developer authentication — Okta OIDC via the gateway

managed settings force gateway login; no local API keys exist



Audit identity chain: Okta identity → session JWT → user.id / user.email / user.groups stamped on every request and telemetry export. Grafana repeats this flow with its own client + strict group→role mapping.

Grafana uses the **same issuer** with its own client and redirect URI (`/grafana/login/generic_oauth`), strict Okta-group→role mapping (`grafana-admins` → Admin; users in no mapped group are denied), PKCE, and the local login form disabled (break-glass `admin` requires a redeploy with `GRAFANA_DISABLE_LOGIN_FORM=false`).

Identity chain for audit: Okta identity → gateway session JWT → every inference request and every telemetry export carries `user.id/user.email/user.groups` . Per-source-IP attribution is deliberately not relied on (ZPA collapses all users to connector IPs — documented in the README).

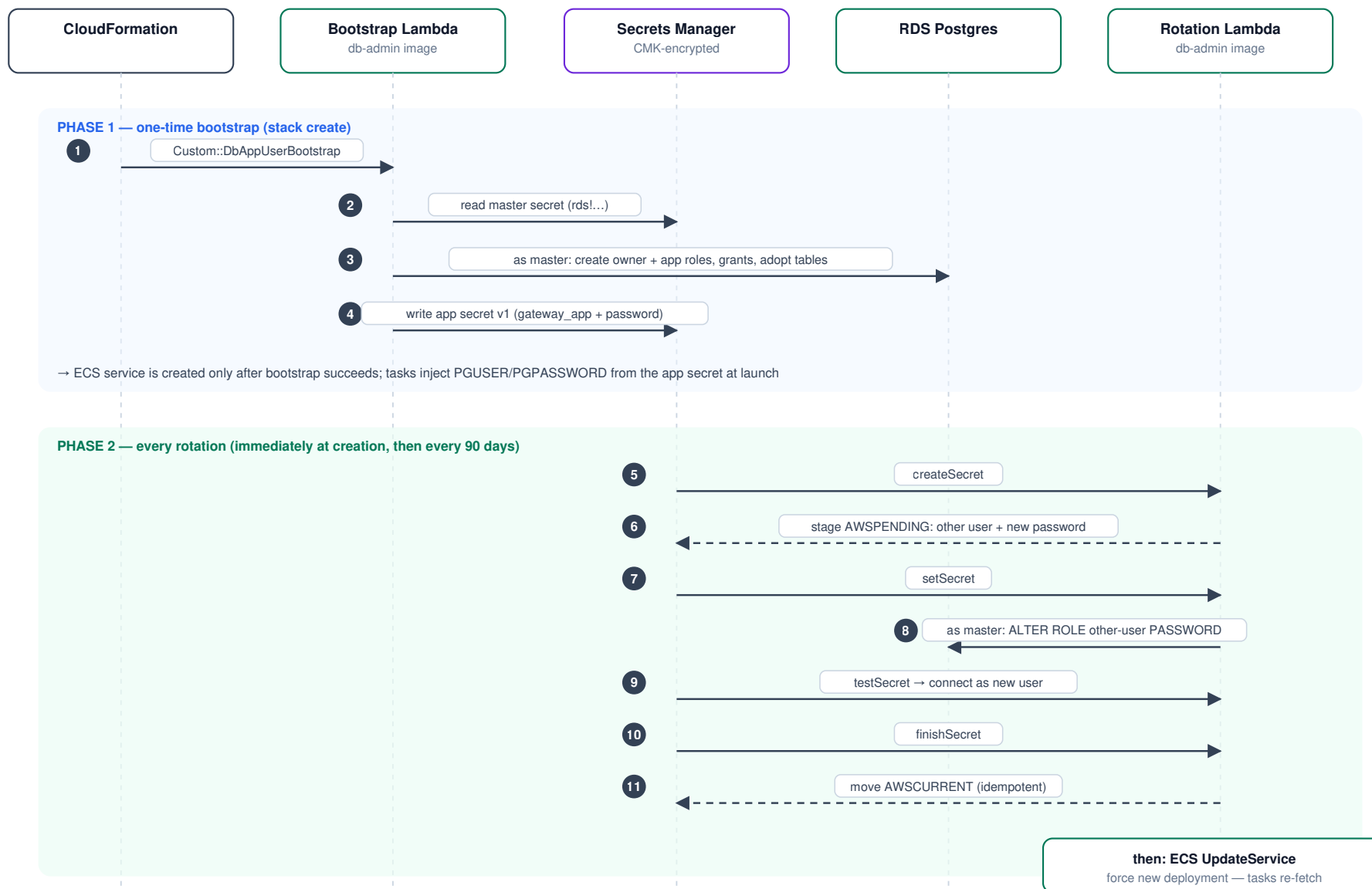
4. Database credential lifecycle

The gateway never holds the RDS master credential (AC-6). Identities:

Postgres role	Kind	Powers	Used by
<code>gw</code> (master)	LOGIN, RDS-managed	<code>rds_superuser</code> -adjacent	Break-glass humans + db-admin Lambda only
<code>gateway_owner</code>	NOLOGIN	Owns schema objects; CONNECT/TEMPORARY/CREATE on the gateway DB only	Assumed at login by both app users
<code>gateway_app</code> / <code>gateway_app_clone</code>	LOGIN (alternating pair)	Via <code>SET role</code> → <code>gateway_owner</code> ; no CREATEROLE, no rds_superuser, cannot alter pgaudit	Gateway tasks (exactly one is live)

Database credential lifecycle — least privilege + alternating-users rotation

the gateway never holds the master credential; the master secret is break-glass only



Why alternating users: the PREVIOUS credential stays valid until the NEXT rotation, so running tasks never hold a dead password — the roll is never time-critical.

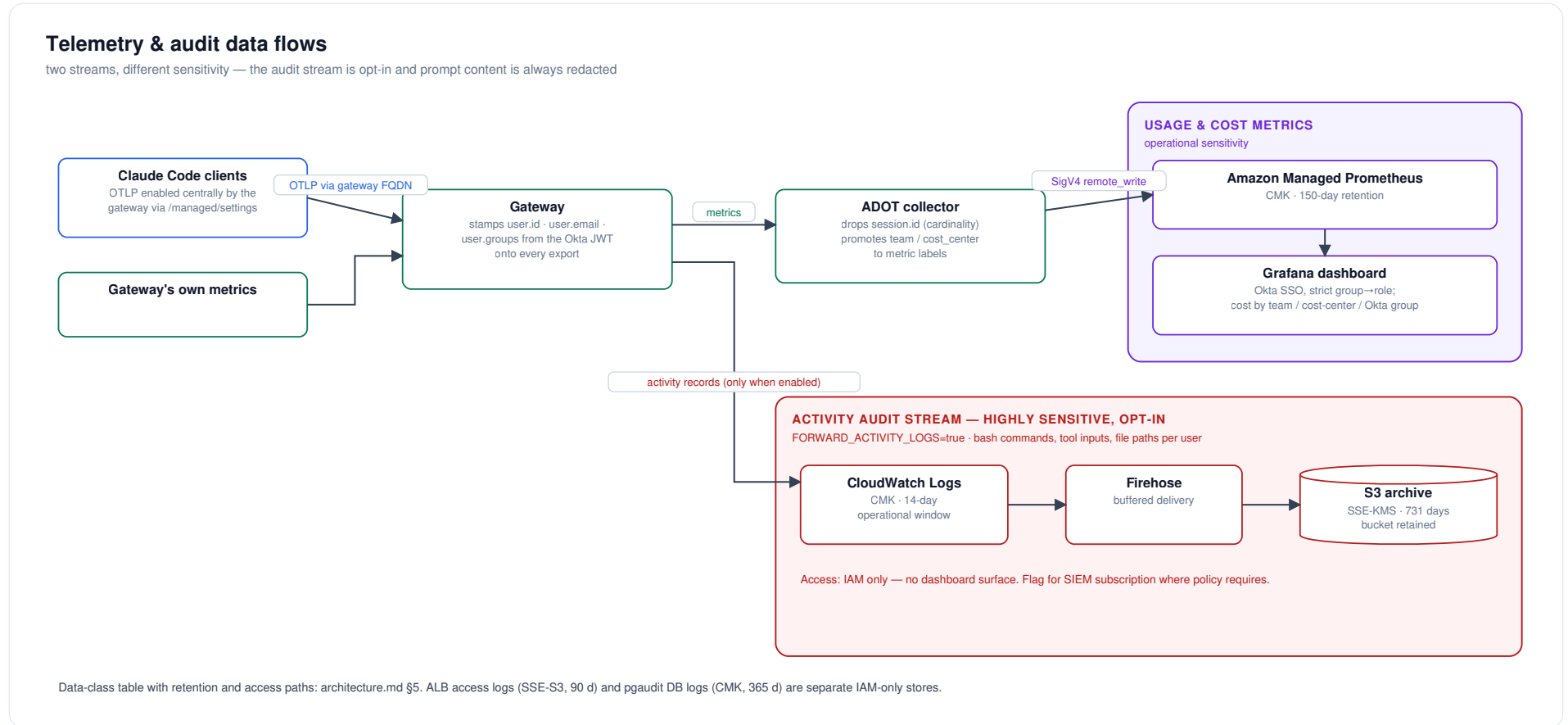
Failure handling: Secrets Manager retries any failed step; a persistently failing rotation trips the <prefix>-db-rotation-errors CloudWatch alarm.

Roles: gw (master, break-glass, RDS-rotated weekly) · gateway_owner (NOLOGIN, owns schema) · gateway_app / gateway_app_clone (LOGIN, SET role → owner; no CREATEROLE, no rds_superuser, cannot touch pgaudit).

Failure containment: a failed rotation step is retried by Secrets Manager; a persistently failing rotation trips the `<prefix>-db-rotation-errors` CloudWatch alarm. The RDS master secret keeps its RDS-managed 7-day auto-rotation, which affects no running workload.

5. Telemetry & audit data flows

Two distinct streams with different sensitivity:



Data class	Contains	Store	Retention	Access path
Usage metrics	tokens, cost, sessions, LoC, model, user identity, team/cost-center	AMP (CMK)	150 d (AMP fixed)	Grafana via Okta SSO, role-mapped
Activity stream (opt-in)	bash commands, tool inputs, file paths, per user; prompt content redacted	CloudWatch (CMK) → S3 (SSE-KMS)	14 d window / 731 d archive	IAM only; flagged for SIEM subscription
ALB access logs	source connector IPs, URLs, timings	S3 (SSE-S3)	90 d	IAM only
DB audit (pgaudit)	DDL, role changes, writes (no bind values)	RDS → CloudWatch (CMK)	365 d group	IAM only
Session/spend store	sessions, spend counters	RDS (CMK)	live	app DB user only

6. Secrets & key inventory

One customer-managed KMS key (`alias/<prefix>`, rotation enabled, created by the DB stack or bring-your-own) encrypts everything below except where noted.

Secret	Store / name	Created by	Consumed by	Rotation
RDS master (gw)	Secrets Manager <code>rds!...</code>	RDS-managed	Humans (break-glass) + db-admin Lambda	RDS-managed, 7 d auto
App DB user	<code><prefix>/db-app-user</code>	Bootstrap Lambda	Gateway tasks (ECS injection at launch)	Stack's rotation Lambda, 90 d default, alternating users + service roll
Okta client secret (gateway)	<code><prefix>/oidc-client-secret</code>	Stack (placeholder) → <code>set-okta-secret.sh</code>	Gateway tasks	Manual, in Okta + script (rolls service)
Okta client secret (Grafana)	<code><prefix>/grafana-oidc-client-secret</code>	Stack (placeholder) → <code>set-grafana-oidc-secret.sh</code>	Grafana task	Manual, same pattern
JWT session-signing secret	<code><prefix>/jwt-secret</code>	Stack (generated)	Gateway tasks	Manual runbook: prepend → roll → remove
Grafana admin password	<code><prefix>/grafana-admin-password</code>	Stack (generated)	Break-glass only (login form disabled)	Manual
TLS: enterprise leaf + key	ACM import	Enterprise CA via <code>import-enterprise-cert.sh</code>	ALB listener	Manual re-import; <code>DaysToExpiry</code> alarm at 30 d
TLS: per-task certs	Generated in-container at startup	Gateway / Grafana endpoints	ALB target connections	Every task launch; keys never leave the task

Handling rules embedded in tooling: secret values never appear on argv (`file://` + mode-600 temp files), never in CloudFormation parameters, and the two `set-* -secret.sh` scripts roll their consuming service.

7. Security-group matrix

Default egress is removed from **every** group (inline rules replace the allow-all); the table is the complete connectivity graph.

Deep dive with diagrams: [network-access-controls.md](#) maps every SG rule pair visually (diagram 7), inventories each group (diagram 8), and traces one API call through all five enforcement layers - SGs, endpoint SG, endpoint policies, IAM, KMS (diagram 9).

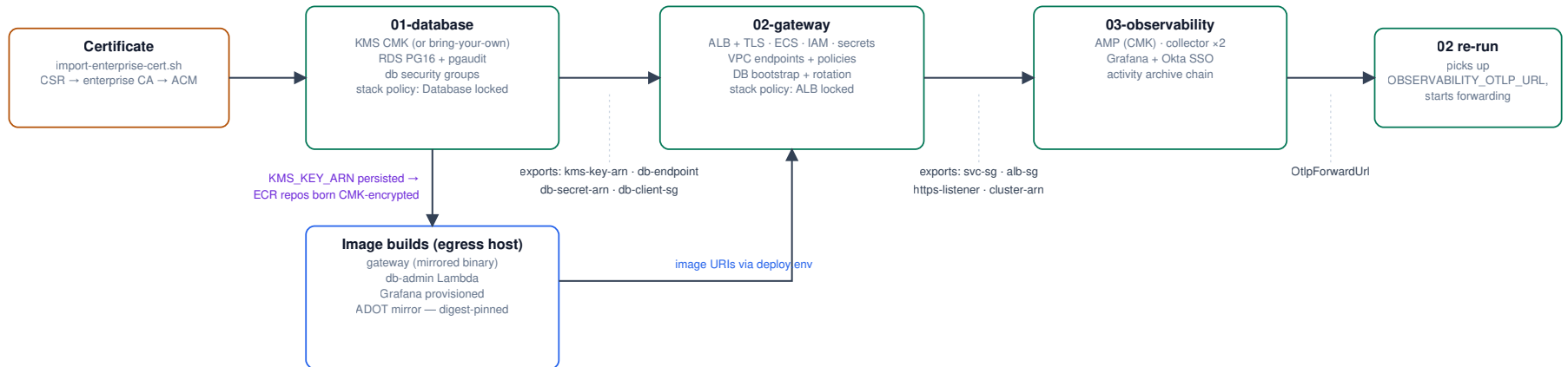
SG	Ingress	Egress
alb	443 from CLIENT_INGRESS_CIDR (ZPA connector subnets)	8080 → svc ; 3000 → grafana (added by 03); otherwise none
svc (gateway tasks)	8080 from alb	443 → 0.0.0.0/0 (Okta + AWS via endpoints/TGW); proxy port when configured; 4317–4318 → collector (added by 03); 5432 via attached db-client
db-client (attached to tasks + db-admin Lambdas)	—	5432 → db only
db	5432 from db-client	none
db-admin (Lambdas)	—	443 → 0.0.0.0/0 (Secrets Manager, ECS APIs)
collector	4317–4318 from svc	443 → 0.0.0.0/0 (AMP, CloudWatch)
grafana	3000 from alb	443 → 0.0.0.0/0 (AMP, Okta); proxy port when configured
endpoints / amp-endpoint	443 from svc / db-admin (resp. collector + grafana)	none

DNS to the VPC resolver is exempt from SG evaluation (AWS platform behavior) and is why no port-53 rules appear.

8. Stack dependencies & deploy order

Stack dependencies & deploy order

arrows carry CloudFormation exports — while imported, an export's value is locked (day-one decisions called out)



Locks a reviewer should know: the RDS storage CMK is fixed at creation (plus 01 ↔ 02 export locks) — a day-one decision; 03 must be deleted before 02 replacement-updates; the ALB and Database carry stack policies denying Update:Replace / Update:Delete.

Full teardown/update ordering: README — Teardown & update order.

Export locks (documented in README "Teardown & update order"): while a downstream stack imports an export, the upstream stack cannot change its value — most notably the RDS storage CMK is a **day-one decision**, and 03 must be deleted before 02 replacement-updates. The ALB and Database additionally carry CloudFormation **stack policies** denying `Update:Replace` / `Update:Delete`, so an accidental template change that would recreate them (new ALB DNS name → client DNS resubmission; new empty database) fails fast.

9. Data-at-rest encryption map

Store	Encryption	Key
RDS storage + snapshots	SSE	CMK
RDS master secret / all Secrets Manager secrets	SSE	CMK
CloudWatch log groups (ECS ×3, activity)	SSE	CMK
Activity archive bucket	SSE-KMS + bucket key	CMK
AMP workspace	SSE	CMK (ENCRYPT_AMP_WITH_CMK , creation-time)
ECR repositories	SSE-KMS at creation	CMK (when created after 01)
ALB access-logs bucket	SSE-S3 (AES-256)	AWS-managed — ELB log delivery does not support KMS

10. Known accepted risks / review notes

Items a reviewer should see up front, with rationale (full history in `security-review-2026-07.md`):

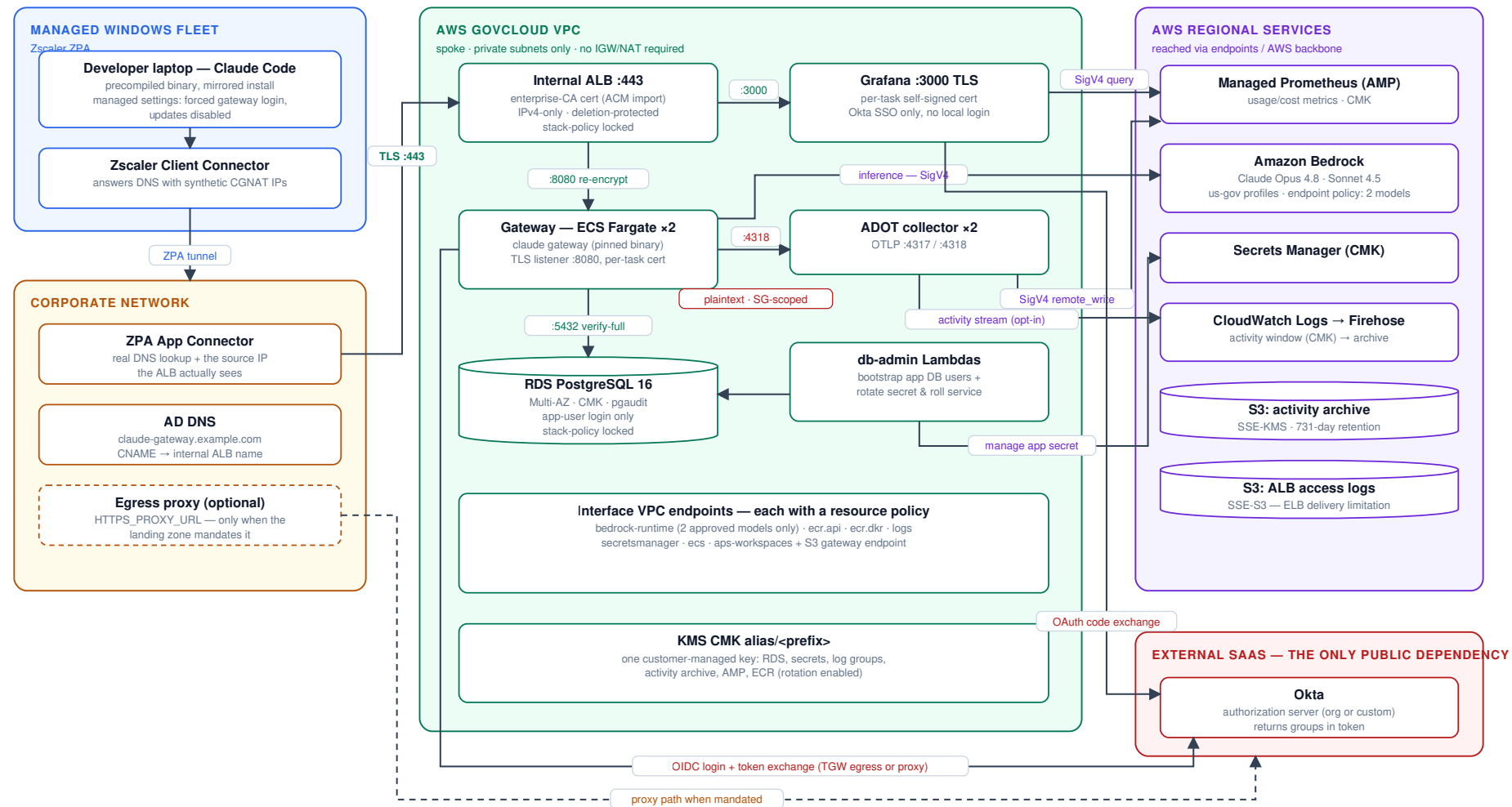
1. **Gateway→collector OTLP hop is plaintext** (§2 hop 6). SG-to-SG scoped (only gateway tasks can connect); VPC traffic is not sniffable by other tenants. Encrypting it requires an enterprise-CA-signed collector cert in a custom collector image + the CA root in the gateway trust store (the gateway validates telemetry TLS against the system store only). Documented recipe on the collector task definition; implement if the SSP rejects the VPC-boundary argument for SC-8.
2. **ALB access logs cannot use the CMK** — AWS platform limitation; SSE-S3 with public access blocked and lifecycle expiry.
3. **S3 Object Lock deferred** by decision (2026-07-15); revisit if AU-9 WORM retention is mandated.
4. **Egress 443 to 0.0.0.0/0 from tasks** — required for Okta (public SaaS, no VPC endpoint, IP ranges not pinnable); mitigated by port scoping, endpoint policies for all AWS traffic, and the central inspection layer in the landing zone.
5. **Foundation-model IAM region wildcard** (`bedrock::*:foundation-model/ <exact-model>`) — required because GovCloud geo inference profiles fan out across us-gov regions; model IDs are exact.
6. **First app-secret rotation is asynchronous** — the stack goes green regardless; verify via the db-rotation log group / errors alarm after first deploy.

Appendix — combined system view

Everything from §1 on a single page — denser than the split views above, useful when a reviewer wants the whole system at a glance.

System architecture & trust boundaries

Claude apps gateway in AWS GovCloud us-gov-west-1 — every arrow is TLS unless flagged



Boundary facts: no public ingress (internal ALB behind ZPA); no public egress from the inference path (Bedrock endpoint, 2-model policy); clients never contact Anthropic (mirrored, verified binaries; updates disabled).